User manual

SparkCalc

SparkCalc computes declared expressions on the live Sparkplug estate and publishes results as their own identity, so a derived value is never mistaken for a measured one.

On the benchIgnition 8.1.19+ and 8.3Runs on: Host

Contents

1. Overview

SparkCalc computes declared expressions on the live Sparkplug estate and publishes results as Sparkplug metrics under their own identity (`Calc-<group>`), so a derived value is never mistaken for a measured one. It is aimed at rates, totals, efficiencies, rollups and unit conversions that today live in a PLC, an Ignition expression tag, or a notebook, none of which make the derived value visible to every non-Ignition consumer of the namespace.

SparkCalc is on the bench, not customer ready, per the module's own README; what is proven and what is owed is documented there.

2. What You Need

- A SparkBridge Host gateway.

3. Install

SparkCalc comes with the Host license. Install it through Config > Modules > Install or Upgrade a Module; no build step is required.

4. License

Included with SparkBridge Host. Covered by the suite license key described in the shared install steps: file at `<data>/sparkbridge-license.key`, re-checked every 30 seconds.

5. Configure

Declared expressions, staleness budgets, and identity settings are documented in the module's own README and in `docs/LIVEPROOF.md`. Only what is confirmed there is listed below; see the README in the bundle for the full property list.

Result identity	Results publish under a `Calc-<group>` identity, with its own edge node and bdSeq lifecycle, distinct from the plant's own identity. The module refuses to produce an identity that could be the plant's.
Staleness budgets	Every declared input has its own staleness budget; there is no single global staleness setting.
Bad-quality behavior	If any input is stale, dead, or never seen, the metric evaluates to nothing: bad quality with no value at all, never zero, never NaN, never the last known value.
Provenance	Every metric's NBIRTH carries `calc.expression`, `calc.inputs`, `calc.stalenessBudgetsMs`, `calc.definitionHash`, `calc.trigger` and `calc.engine`. Every NDATA carries `Quality`, `calc.quality`, `calc.stalestInput`, `calc.ageMs`, and when bad, `calc.reason`.

6. Operate

A computed metric's timestamp is always the stalest contributing input's source timestamp, never the evaluation clock: a ratio of a flow read at 12:00:00 and a level read at 11:59:55 describes the plant as of 11:59:55. When any declared input goes stale, dead, or was never seen, the metric publishes bad quality with no value at all rather than a stale or default value. Because the calculation engine is a pure function of its inputs and its own threaded state, a derived series can be recomputed over history, not only observed forward from now.

7. Verify

Confirm published metrics appear under the `Calc-<group>` edge node rather than under the plant's own edge, and that the reported timestamp is the stalest contributing input's source timestamp, not the evaluation time. Full proof steps are in `docs/LIVEPROOF.md` in the repo.

8. Troubleshoot

Symptom	Cause	Fix
Metric never publishes a value	One or more declared inputs are stale, dead, or never seen	Check the input's own staleness budget and confirm it is publishing on the expected schedule; SparkCalc will not substitute a default
Computed value appears under the wrong edge node	Should not occur: the module refuses any identity that could collide with the plant's own	If seen, file it as a defect; this is a designed invariant, not a configuration option
Timestamp on a computed metric looks wrong	Normal: the timestamp is the stalest contributing input's timestamp, not now	Check `calc.stalestInput` and `calc.ageMs` on the metric to see which input is holding it back

9. Reference

- Result identity prefix: `Calc-<group>`.
- Full config key list and expression surface: module README and `docs/LIVEPROOF.md` in the bundle.

10. Support

SparkCalc in the docs · SparkCalc on the Host product page · Contact · GitHub releases